

M2 IKSEM Session 1 – Finals IT4-Blockchain

Nicolas Herbaut. *Duration: 90 minutes*

MIAGE, UFR27, Université Paris 1 Panthéon-Sorbonne 2022-2023

1 Blockchain concepts (10 pts)

1.1 Blockchain and technology

1.1.1 Bitcoin (2pt)

Why do we say that bitcoin offers probabilistic finality? How is it fixed in permissioned blockchain?

1.1.2 Ethereum PoS (2pt)

Explain 3 main advantages of PoS over PoW.

1.1.3 DAO (2pt)

Do you think it was a good idea to create a hard fork in Ethereum after TheDAO exploit (no wrong answer here)? Explain your point of view.

1.2 Legal Aspects

1.2.1 ICO (2pt)

What is an Initial Coin Offering (ICO)? Why did regulation appear fast on the subject?

1.2.2 Blockchain and regulation (2pt)

Why is regulation important for blockchain systems? And why should it be incorporated?

2 Blockchain Patterns (10 pt)

2.1 Blockchain for MIAGE attendance list

Miage students that choose the apprenticeship path must follow all the courses at the university, since they are paid by the company that employ them. Today, attendance lists are signed by the student and the professor, transmitted to the secretary at the end of the course, scanned to pdf files to be used as legal proofs and a semestral spreadsheet containing the absence data is manually filed by Stephane and sent to the CFA. With this spreadsheet, companies can withhold the unjustified absence from the apprentice salary. Absence can only be justified with a physician's certificate, transmitted to the secretary and stored along the scanned attendance lists for legal purpose.

Since the process is mostly manual, it is error-prone and time consuming. Note that today, the university and companies interact only through the CFA.

2.2 Questions

2.2.1 Advantages of a blockchain solution

As a blockchain consultant, you take the same elevator as Laetitia Van Bloeme (MBA in marketing and project manager at AFIA CFA) to reach the 14th floor of PMF. Since some social science students are still in the building, you stop at each floor, so you have 3 minutes to explain how the deployment of a blockchain system common to MIAAGE, AFIA and the companies that employ MIAAGE students could streamline the attendance list production and save time on the salary withholding process.

2.2.2 Solution Diagram

Your elevator speech has paid off: you now have an appointment with Yves Georgelin (Automation Engineer from Mines Douai, a minor French engineering school not to be confused with MINES ParisTech and director of AFIA CFA) to explain how to implement the solution you described to Laetitia Van Bloeme.

Using a component diagram, show how to this solution can be deployed using battle-tested patterns, such as the ones you've seen during your beloved Blockchain course which are summarized in the cheat sheet provided. You must specify which blockchain technology should be used.

Describe the diagram (i.e., the components and their relationship) to better explain how your proposal works.

2.2.3 List the key patterns

List the 3 key patterns used in your implementation. For each pattern, explain how and why it is used. Make sure to justify pattern use in the context in which it is used, not only the intrinsic properties of the pattern.

3 Cheat Sheet

3.1 Interaction with External

These patterns solve problems related to the interaction between blockchain-based apps and external systems, such as retrieving reliable external data, integrating with existing systems, and managing events.

3.1.1 Oracle Pattern

Allows a BC to interact with external data sources (e.g., weather, USD/ETH value). The oracle possesses a private key to publish information from the external world to the BC.

3.1.2 Reverse Oracle

Enables external components of an existing system to request data using a BC and smartCons. Makes BC data available to external components.

3.1.3 Legal & SC Pair

Addresses the secure execution of digital agreements by providing a pair of contracts, a legal contract and a smartCon, that work together to enforce the conditions of a digital agreement, ensuring reliable and transparent execution. A contractual document with conditions would have a corresponding smartCon representation on the BC. The 2 contracts contain links to each other's version, and there may be off-chain conditions (e.g., "I have paid the contract"). Ex: in real estate, a legal contract can be used to define the terms and conditions of a lease agreement, while a smartCon can automatically execute rent payments and security deposits based on the conditions specified in the legal contract.

3.2 Data Management

These patterns solve problems related to data management in BC-based apps, such as storing internal state for facilitating future transactions, digitally representing physical or virtual assets, and controlling access to data stored on the BC.

3.2.1 Encrypting On-Chain Data

Ensures the confidentiality of data stored on the BC by encrypting it. People exchange symmetric decryption keys to read the data.

3.2.2 Tokenization

Enables the digital representation of physical or virtual assets on the BC (colored coins). Provides proof of ownership of a particular asset in the real world. Useful for distribution purposes.

3.2.3 Off-chain Data Storage

Uses hashing to ensure the integrity of arbitrarily large data sets that cannot be directly placed on the BC. The actual data is stored off-chain, ex on a cloud, while only the hash of the data is stored on the BC.

- when signing a contract, the hash of the contract is calculated and stored on the BC (which is replicated multiple times). To verify that the document has not been altered, one simply needs to check if its hash still matches the one stored on the BC.
- an online voting app where voters can submit their votes using a hash created off-chain to link their authority to their vote, allowing anonymous voting while ensuring the authenticity and authorization of each vote.

3.2.4 State Channel

Improves scalability by storing intermediate states of micro-payments off-chain and finalizing transactions on the BC, storing only finalized payments on the BC.

- Ex: in an online gaming app, players can make micro-transactions to purchase virtual items. The transactions would occur off-chain, but the players' balances would be updated on the BC only when they exit the game or make a significant purchase. In the case of Wi-Fi billing in a hotel charged per Mo, instead of sending multiple transactions, the transaction is updated incrementally before being sent to the BC. An alternative approach is to have a contract on which everyone can interact (but this incurs costs for every change). The challenge is to have trust in the communication channel before the data is sent.

3.3 Security

These patterns solve problems related to security in BC-based apps, such as transac-

tion validation, identity management, privacy protection, and transaction history tracking.

3.3.1 Multiple Authorization

Addresses the security of updates on a BC by requiring authorization from multiple addresses before an update can be executed, ensuring that updates can only be performed by authorized parties. An operation is executed if a certain num of people among all present parties agree to it.

3.3.2 Off-chain Secret Enabled Dynamic Authorization

Uses a hash created off-chain to dynamically link the authority of a transaction.

- Ex: in an online voting app, voters can submit their votes using a hash created off-chain to link their authority to their vote, allowing anonymous voting while ensuring the authenticity and authorization of each vote. Someone creates contracts with a lock that can only be opened with keys possessed by others. However, at the time of locking, they haven't defined who will have the key. In essence, it is not necessary to know who will be given the key at the time of creating the smartCon. The key can only be used once.

3.3.3 X-Confirmation

Waiting for a sufficient num of blocks as confirmation to ensure that a transaction added to the BC is immutable with a high probability.

3.4 Structural Patterns of Contract

3.4.1 Contract Registry

Addresses the updating issue of smartCons on a BC by allowing the location of the latest version's address through a contract registry lookup before invocation.

- Ex: in a marketplace app, sellers can deploy smartCons to sell their products, and buyers can invoke these contracts using the contract name to purchase the products.
- Note: The object is final, but its attributes are not. The object deletes

data and code when the contract is updated.

3.4.2 Embedded Permission

Addresses the access permission management issue in a smartCon by incorporating authorization rules directly into the contract's code, enabling control over access to data stored on the BC (verification is done using the public key of the person). Before the execution of certain functions, the identity of the executing parties is verified. Verification is done to check if the public key corresponds to the individuals who are authorized to interact with specific functions of the smartCon.

3.4.3 Data Contract

Stores data in a separate smartCon. When used with Contract Registry, it allows for the updating of the smartCon while preserving the data without modifying the code.

3.4.4 Factory Contract

Enables the creation of instances of smartCons with custom parameters. The factory contract is immutable, ensuring security and simplicity of use. However, it is slightly more expensive than deploying (cost of the factory plus creating a contract from the factory instead of deploying from the source code).

- Ex: in a healthcare app based on the BC, this pattern has been used to create customized smartCons for each patient, enabling efficient management of medical records and better coordination among healthcare professionals.

3.4.5 Incentive Execution

Addresses the issue of user motivation to execute smartCon functions by offering rewards for invoking and executing functions, encouraging active participation in the BC app. Normally, a person who is asked to create a function on a smartCon incurs costs. The goal is to reimburse them after calling a function. They call a function that grants them the right to call another function that will provide them with codes.